

Code Layout, Readability and Reusability

This document evaluates the code quality standards applied in OptiCrop, covering naming conventions, modularity, documentation, and code style.

1. Code Layout Principles

- " **PEP 8 Compliance:** All Python files pass flake8 linting with max-line-length=100
- " **Indentation:** 4 spaces (no tabs) throughout all .py files
- " **Imports:** Grouped as stdlib third-party local; sorted alphabetically within groups
- " **Blank lines:** Two blank lines between top-level functions/classes; one within classes
- " **File length:** No single file exceeds 300 lines; logic split across app/ml/ modules

2. Naming Conventions

Scope	Convention	Example
Variables / functions	snake_case	validate_inputs(), crop_label, confidence_score
Classes	PascalCase	CropPredictor, InputValidator
Constants	UPPER_SNAKE_CASE	MODEL_PATH, FEATURE_NAMES, FEATURE_RANGES
HTML templates	snake_case filenames	index.html, result.html, about.html
CSS classes	BEM-inspired kebab-case	.prediction-card__crop-name

3. Docstring Standard (Google Style)

Every public function and class includes a Google-style docstring:

```
def predict_crop(features: np.ndarray) -> tuple[str, float]:
    """Predict the optimal crop given a scaled feature vector.

    Args:
        features: 1D numpy array of shape (7,) scaled soil/climate values.

    Returns:
        Tuple of (crop_label: str, confidence: float) where confidence is 0-1.

    Raises:
        ModelNotLoadedError: If model.pkl is missing or corrupted.
    """
```

4. Modularity & Reusability

Module	Reusability Design
app/ml/predictor.py	Standalone module importable by CLI, REST API, or future mobile backend
app/ml/preprocess.py	Stateless functions validate_inputs() and scale_features() independently testable
models/ dir	Model artefacts decoupled from code swap model.pkl without changing app logic
templates/base.html	Jinja2 base with {% block content %} all pages extend one base template
static/css/style.css	Single custom stylesheet Bootstrap overrides isolated in one file

5. Code Review Checklist

Check	Status	Evidence
flake8 passes (0 errors, 0 warnings)	PASS	CI log: 0 issues on main branch
Black formatter applied	PASS	Pre-commit hook; Black==24.x
All functions have docstrings	PASS	Reviewed in code walkthrough session
No hardcoded file paths	PASS	os.path.join with CONFIG constants
No bare except clauses	PASS	Specific exceptions caught and logged
Unit tests for all utils functions	PASS	17 tests in tests/; 100% pass rate
No commented-out dead code	PASS	Removed before final commit
requirements.txt pinned to exact versions	PASS	pip freeze > requirements.txt